# PRO2000 - PROGRAMMING

## Week 38- Arrays, strings, structures

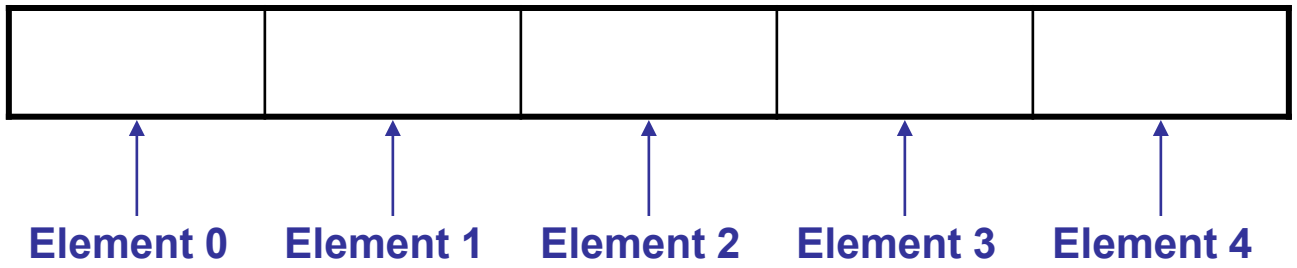
Gebremariam Assres
Gebremariam.assres@kristiania.no

# Today's topics

- Arrays
  - Arrays as Function Arguments
  - Multi-Dimensional Arrays
  - Vectors
- Strings
- Structures

# Arrays

- Array: variable that can store multiple values of the same type
- Values are stored in consecutive memory locations
- Declared using **[]**operator **const int ISIZE = 5; int tests[ISIZE];**
- Allocates the following memory



# Arrays ...

In the definition `int tests[ISIZE];`

- **`int`** is the data type of the array elements
- **`tests`** is the name of the array
- **`ISIZE`**, in **`[ISIZE]`**, is the size declarator. It shows the number of elements in the array.
- The size of an array is the number of bytes allocated for it

*(number of elements) * (bytes needed for each element)*

# Arrays ...

- Examples: Assumes **`int`** uses 4 bytes and **`double`** uses 8 bytes

```
const int ISIZE = 5, DSIZE = 10;
int tests[ISIZE]; // holds 5 ints, array
                  // occupies 20 bytes
double volumes[DSIZE];// holds 10 doubles,
                      // array occupies
                      // 80 bytes
```

# Arrays …

- Can be initialized during program execution with assignment statements

```
tests[0]=79;
tests[1]=82;//etc.
```

- Can be initialized at array definition with an initialization list

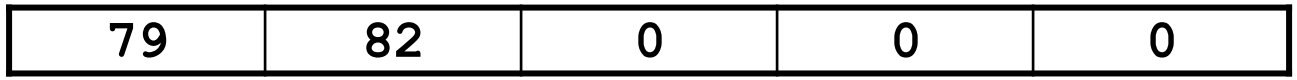
```
const int ISIZE = 5;
int tests[ISIZE] = {79,82,91,77,84};
```

- Implicitly declaration can determine array size by the size of the initialization list-

```
short quizzes[]={12,17,15,11};
```

- If array is partially initialized as fewer values than declared, remaining elements will be set to 0 or the empty string

```
int tests[ISIZE] = {79, 82};
```

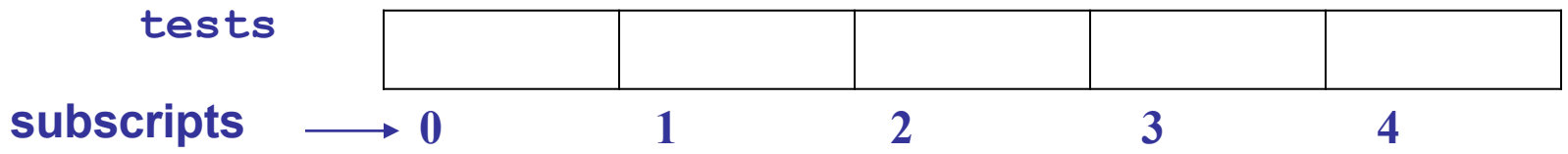


# Accessing elements

- Each array element has a subscript, used to access the element- subscripts start at 0
- Array elements (accessed by array name and subscript) can be used as regular variables

```
tests[0] = 79;
cout << tests[0];
cin>> tests[1];
tests[4]=tests[0]+ tests[1];
cout<<tests;  //illegal- no subscript
```

**cin>>tests[3];** //int constant
**cout<<tests[i];** //int variable
**cout<<tests[i+j];** //int expression



# Accessing elements …

- To access each (all) element of an array
  - Use a loop - let the loop control variable be the array subscript
  - A different array element will be referenced each time through the loop

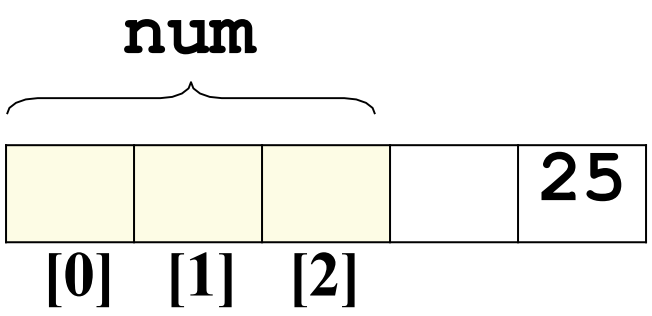
```
//Display elements
for (i = 0; i < 5; i++)
cout<<tests[i]<<endl;
```

```
//Getting array data from a file
const int ISIZE = 5, sales[ISIZE];
ifstream dataFile;
datafile.open("sales.dat");
if (!dataFile)
     cout << "Error opening data file\n";
else{ // Input daily sales
     for (int day = 0; day < ISIZE; day++)
     dataFile >> sales[day];
     dataFile.close();
}
```

# Accessing elements ...

- There are no checks in C++ that an array subscript is in range
- An invalid array subscript can cause program to overwrite other memory
- Example:

```
const int ISIZE = 3;
int i = 4;
int num[ISIZE];
num[i] = 25;
```



# Accessing elements ...

- Array elements can be as ordinary variables in arithmetic operations, in relational expressions, etc.

```
if (principalAmt[3] >= 10000)
        interest = principalAmt[3] * intRate1;
```

- When using ++ and -- operators, don't confuse the element with the subscript

```
tests[i]++; // adds 1 to tests[i]

tests[i++]; // increments i, but has no effect on tests
```

# Accessing elements …

- To copy one array to another - use element-by-element

```
tests2 = tests; //won't work
for (int indx=0; indx < ISIZE; indx++) tests2[indx] = tests[indx];
```

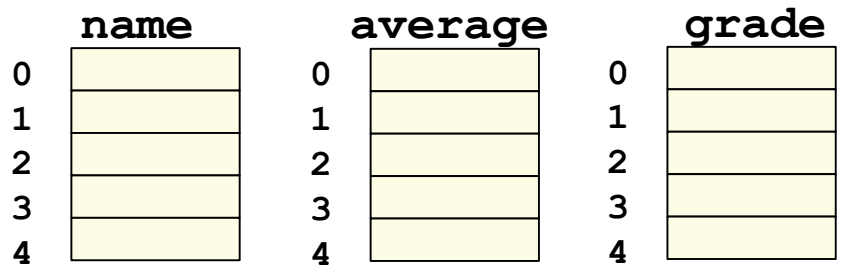
- Partially filled arrays:
  - Estimate maximum capacity and use a sentinel value to indicate the end of data.
  - Track the number of array elements currently in use.
- When to use arrays:
  - Use simple variables when data is processed only once, such as finding sums, averages, minimums, or maximums.
  - Use arrays when data must be processed multiple times, such as comparing values with their average.

# Accessing elements ...

- Parallel arrays: two or more arrays that contain related data
- Subscript is used to relate arrays- elements at same subscript are related and they do not have to hold data of the same type

```
const int ISIZE = 5;
string name[ISIZE];// student name float
average[ISIZE]; // course average char
grade[ISIZE];// course grade

...
for (int i = 0; i < ISIZE; i++)
cout<<"Student:"<<name[i]
      <<"Average:"<<average[i]
      <<"Grade: "<<grade[i]
      << endl;
```



# Accessing elements …

- Use **`typedef`** statement to create an alias for for arrays and to make code more readable like **`typedef`** ***`existingType newName`***`;`

```
typedef unsigned int Uint;
Uint tests[ISIZE]; //array of unsigned ints

// Define yearArray as a data type that is an array of 12 ints
typedef int yearArray[MONTHS];

// Create two of these arrays
yearArray highTemps, lowTemps;
```

# Arrays as function arguments

- Passing a single array element to a function is no different than passing a regular variable of that data type

```
displayValue(score[i]);// call

void displayValue(int item) // header
{cout << item << endl;}
```

- To define a function that has an array parameter, use empty `[]` to indicate the entire array argument- use array name to pass to a function

```
void showScores(int []); // Function prototype
void showScores(int tests[]) // Function header
showScores(tests); // Function call & can also pass size
```

- Array parameters allow functions to modify the original array. Use const to prevent unintended changes when passing arrays to functions.

# Multi-dimensional arrays

- Can define one array for multiple sets of data like a table in a spreadsheet- using two size declarators in definition
- Two-dimensional arrays are initialized row-by-row

```
int exams[2][2] = { {84, 78},{92, 97} };
```

| | |
|---|---|
| 84 | 78 |
| 92 | 97 |

- Use two subscripts to access element `exams[2][2] = 86`
- Use array name and number of columns as arguments in function call `getExams(exams, 2);`
- Use empty `[]` for row and a size declarator for col in the prototype and header

```
void getExams(int[][NUM_COLS], int);//Prototype
void getExams(int exams[][NUM_COLS],int rows) // Header
```

# Multi-dimensional arrays ...

- To traverse 2D array, use nested loops, one for row and one for column, to visit each array element.
- Accumulators can be used to sum the elements row-by-row, column-by-column, or over the entire array.
- Can define arrays with any number of dimensions

```
short rectSolid(2,3,5);
double timeGrid(3,4,3,4);
```

- When used as parameter, specify size of all but 1st dimension

```
void getRectSolid(short [][3][5]);
```

# Vectors

- Defined in the Standard Template Library (STL)
- Holds a set of elements, like an array but number of elements can flexibly grow and shrink
  - No need to specify size when defined
  - Automatically adds more space as needed
  - Can use [] to access elements
- Must `#include <vector>` header file to use vectors
- Hold values of any type specified when a vector is defined

```
vector<int> scores; vector<double> volumes;
```

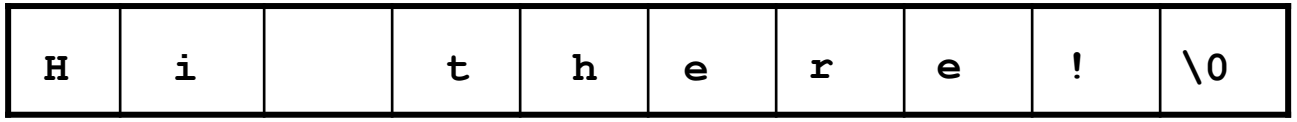
# Vectors ...

- Define a vector of integers (starts with 0 elements): `vector<int> scores;`
- Define `int` vector with initial size 30 elements: `vector<int> scores(30);`
- Initialize all elements to 0: `vector<int> scores(30, 0);`
- Use `push_back` member function to add an element to a full array:
  `scores.push_back(75);`
- Use `size` member function to determine number of elements currently in a vector `howbig = scores.size();`
- Use `pop_back` member function to remove last element from vector
  `scores.pop_back();`
- To remove all contents of vector, use `scores.clear()` member function
- To determine if vector is empty, use `empty` member function
  `while (!scores.empty()) ...`

# Strings

# C-Strings

- C-string is sequence of characters stored in adjacent memory locations and terminated by **`NULL`** character
- The C-string "Hi there!" would be stored in memory as shown:




- The NULL character is used to indicate the end of a string
- It can be specified as the character **`'\0'`**, the **`int`** value 0, or the named constant **`NULL`**
- C-strings can be represented as string literal **`"Hi There!"`**, pointer **`char *p`**, array **`char str[20]`** all as pointers to char

# C-Strings ...

- A string literal is stored as a null-terminated array of **`char`**- address of first char (pointer) as the value of the string
- An array of char can be defined and initialized to a C-string

```
char str1[20] = "hi";
```

- An array of char can be defined and later have a string copied into it using **`strcpy`** or **`cin.getline`**

```
char str2[20], str3[20];
strcpy(str2, "hi");
cout << "Enter your name: ";
cin.getline(str3, 20);
```

- C-string defined as an array can be referred to in other parts of the program by using the array name.

# C-Strings ...

- Pointer to **`char`** defined as `char *pStr;`
- Useful in repeatedly referring to string literal

```
pStr = "Hi there"; cout << pStr << " "<<pStr;
```

- Pointer to **`char`** can also refer to C-strings defined as arrays of char

```
char str[20] = "hi";
char *pStr = str;
cout << pStr;     // prints hi
```

- Can dynamically allocate memory to be used for C-string using **`new`**

# C-String library

- It requires `cstring` header file and functions take C-strings as arguments- array of char, pointer to char or string literal
  - `strlen` returns length of a C-string: `cout<<strlen("hello");` prints `5`.
  - `strcat` concatenates two strings: `strcat(str1, str2);`
  - `strcpy (copy), strcmp (compare)`
  - `strstr(str1,str2)` searches for the occurrence of str2 within str1.
  - `atoi` converts **a**lphanumeric **to** **i**nt
  - `atol` converts **a**lphanumeric **to** **l**ong
  - `atof` converts a numeric string to a float
- Character Testing- require `cctype` header file
  - `isalpha` **true** if arg. is a letter, **false** otherwise
  - `isalnum` **true** if arg. is a letter or digit, **false** otherwise
  - `isdigit` **true** if arg. is a digit 0-9, **false** otherwise
  - `islower` **true** if arg. is lowercase letter, **false** otherwise

# C++ `string` class

- The string class offers advantages over C-strings- has more functions, overloaded operators → simplified expressions

- Need to include the `#include<string>` header file
- Can be processed using array name, entire string at once, or one element at a time by using a subscript in [].
- String class constructors:
  - Default constructor `string()`
  - Copy constructor `string(string&)` initializes with values of other string objects
  - Convert constructor `string(char *)` initializes string objects with values of C-strings
  - Various other constructors

# C++ `string` class ...

- Overloaded string operators

| Operator | Meaning |
|---|---|
| `>>` | Reads whitespace-delimited strings into string object |
| `<<` | Inserts string object into a stream |
| `=` | Assigns string on right to string object on left |
| `+=` | Appends string on the right to the end of contents of string on left |
| `+` | Returns concatenation of the two strings |
| `[]` | references character in string using array notation |
| `>, >=,<, <=, ==, !=` | relational operators for string comparison. Return **true** or **false** |

# C++ `string` class …

- Overloaded string operators

```
string word1, phrase;
string word2 = " Dog";
cin >>word1; // user enters "Hot", word1 has "Hot"
phrase=word1 +word2; //phrase has "Hot Dog"
phrase +=" on a bun";
for (int i = 0; i < 16; i++) cout<<phrase[i]; //displays
```

- **`string`** Member Functions:
  - conversion to C-strings: `c_str`, `data`
  - modification: `append`, `assign`, `clear`, `copy`, `erase`,`insert`, `replace`, `swap`
  - space management: `capacity`, `empty`, `length`,`resize`, `size`
  - substrings: `find`, `substr`
  - comparison: `compare`

# C++ **`string`** class …

- Conversion to C-strings
  - **`data()`** and **`c_str()`** both return the C-string equivalent of a **`string`** object
  - Useful when using a string object with a function that is expecting a C-string

```
char greeting[20] = "Have a ";
string str("nice day");
strcat(greeting, str.data());
```

- Modification of **`string`** objects:
  - appends contents of a **`string`** to its end. **`string str("Have a "); str.append("nice day");`**
  - **`str.insert(int pos, string s)`** **inserts s at position pos in str.**

```
string str("Have a day");
str.insert(7, "nice ");
```

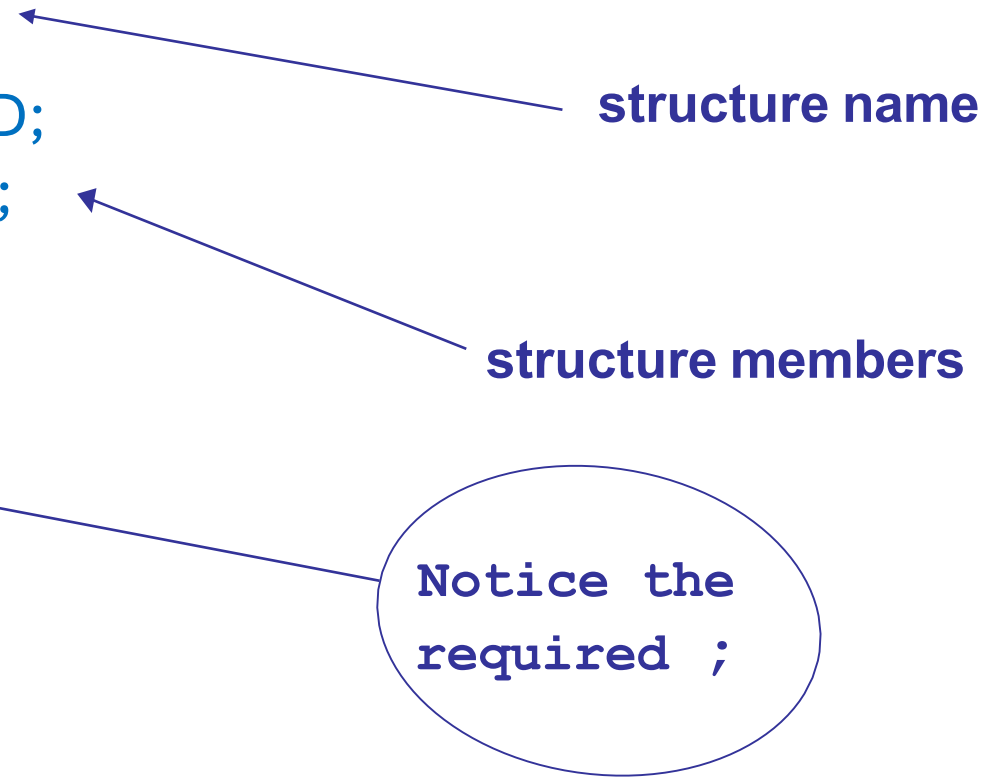
# Structures

# Structures

- Structure is programmer-defined data type that allows multiple variables to be grouped together and declaration format:

```
struct name {
    type1 field1;
    type2 field2;
    ...
    typen fieldn;
};
```

```
struct Student{
    int studentID;
    string name;
    short year;
    double gpa;
};
```



# Structures ...

- **`struct`** declaration does not allocate memory or create variables, no intialization
- To define variables, use structure tag as type name
- Structures can be used as array elements, and use array subscript to access a specific structure in the array.
- Then use dot operator to access members of that structure

```
const int CSIZE = 30;
Student class[CSIZE];//Holds 30 Studen structures
cin  >> class[25].studentID;
cout << class[i].name << " has GPA "
<< class[i].gpa << endl;
```

# Structures ...

- To compare structs, compare member variables:

```
if (s1.gpa >= s2.gpa) // better
```

- Structure members are initialized at the time a structure variable is created with either
  - an ordered initialization list in {}
  - a constructor defined in struct

```
Student (int ID; …){studentID=ID; …}
```

# Structures ...

- Nested structures- having another structure as a member.

```
struct PersonInfo{
   string name,
   address, city;
};
struct Student{
   int studentID;
   PersonInfo  pData;
   short year;
   double gpa;
};
```

- Use the dot operator multiple times to access fields of nested structures

```
Student s5;  s5.pData.name = "Joanne";
s5.pData.city = "Tulsa";
```

# Structures ...

- Structures as function arguments
  - May pass members of **`struct`** variables to functions
    `computeGPA(s1.gpa);`
  - May pass entire **`struct`** variables to functions `showData(s5);`
  - Can use reference parameter if function needs to modify contents of structure variable
- Returning a structure from a function
  - Function can return a struct

```
Student getStuData(); // prototype
s1 = getStuData(); //call
```

  - Function must define a local structure variable for internal use /to use with return statement

# Structures ...

- Example of returning a structure

```
Student getStuData()
{ Student s;        // local variable
   cin >> s.studentID; cin.ignore();
   getline(cin, s.pData.name);
   getline(cin, s.pData.address);
   getline(cin, s.pData.city); cin >>
   s.year;
   cin >> s.gpa;
   return s;
}
```

# Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on Arrays, strings, structures*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)